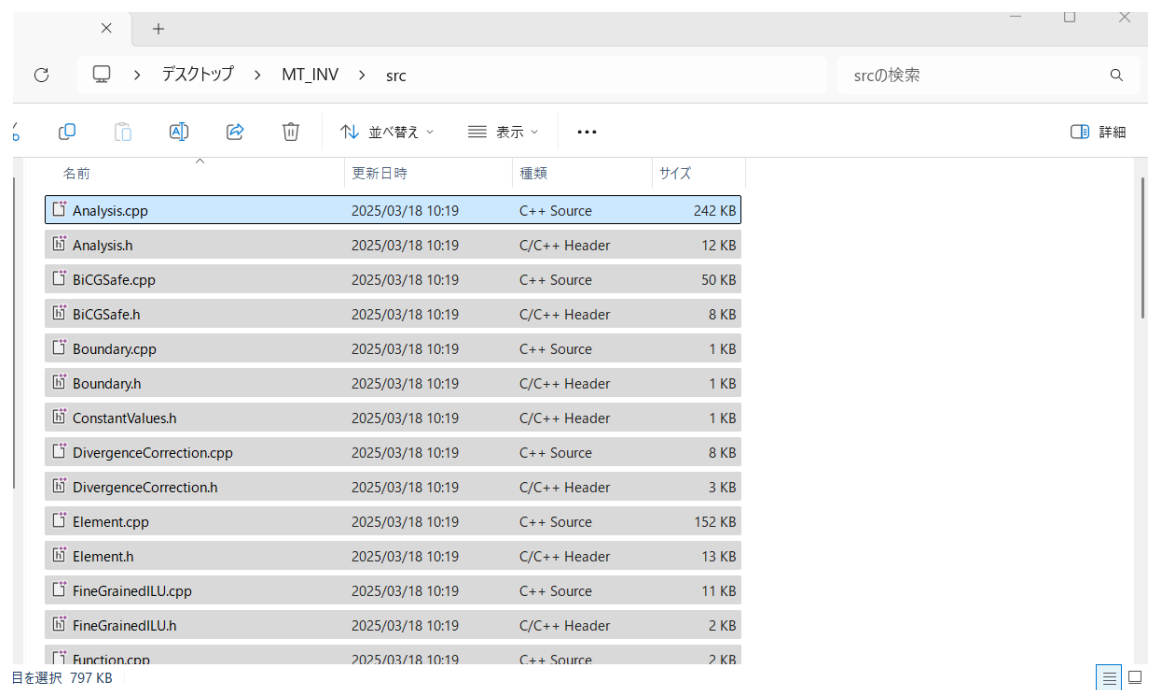


1 必要なプログラム・ソースコードの用意

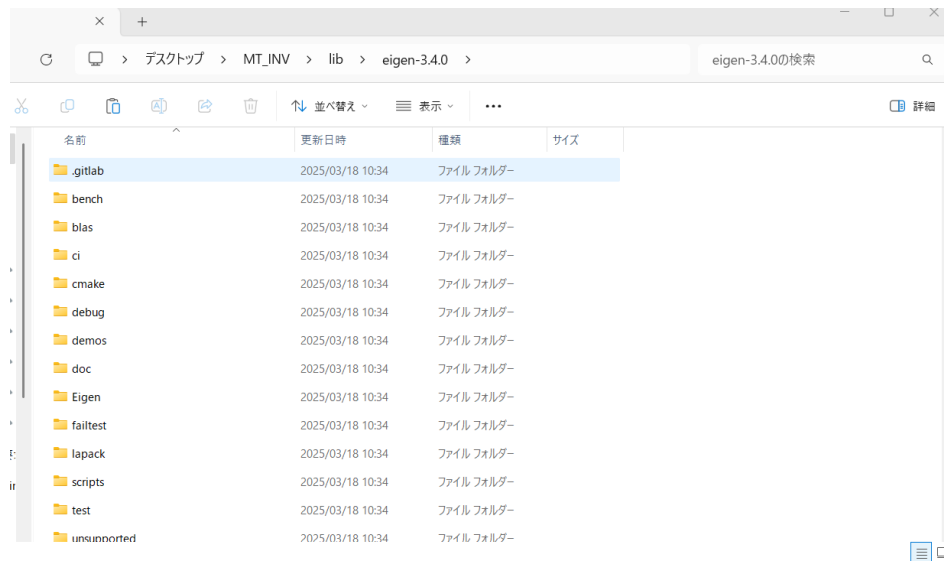
- ・ 以下にアクセスし計算コードを適当なフォルダにダウンロード

<https://github.com/SuzukiAtsushi19911107/FV3DMT>



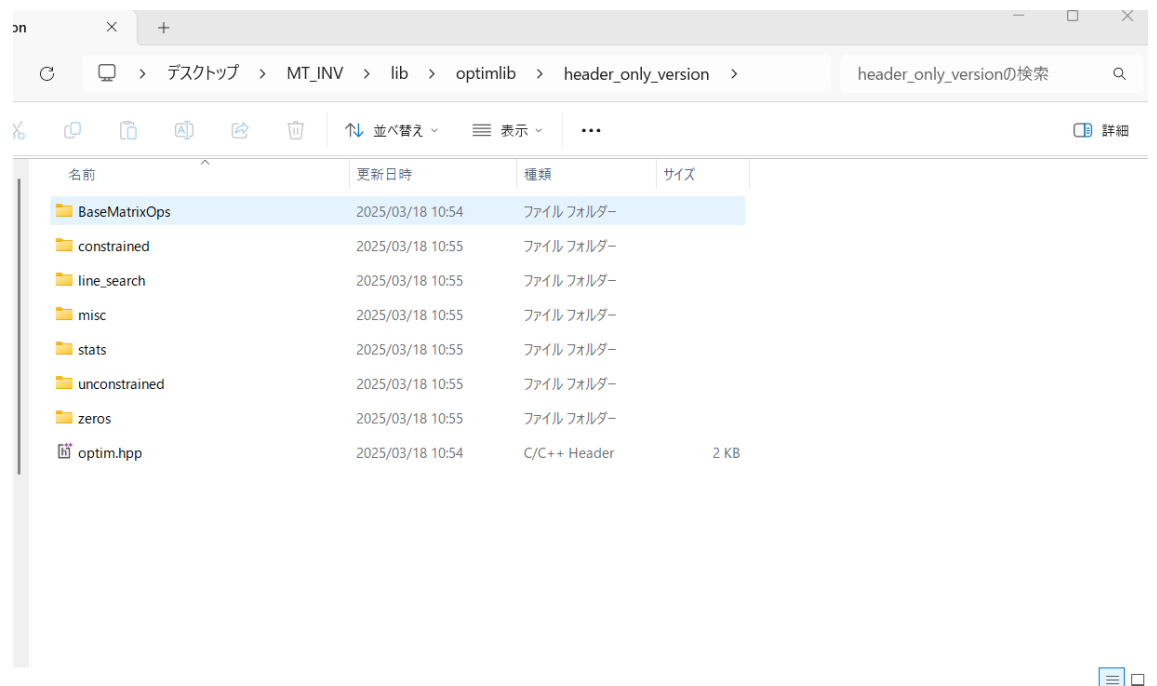
- ・ Eigen を以下から適当なフォルダにダウンロード

<https://eigen.tuxfamily.org/>



- ・ 計算コードの中に、optimlib というフォルダがあるので、適当な場所に置く。

(注意：：OptimLib をオリジナルからダウンロードすると 2023 年 3 月現在で動かない！)



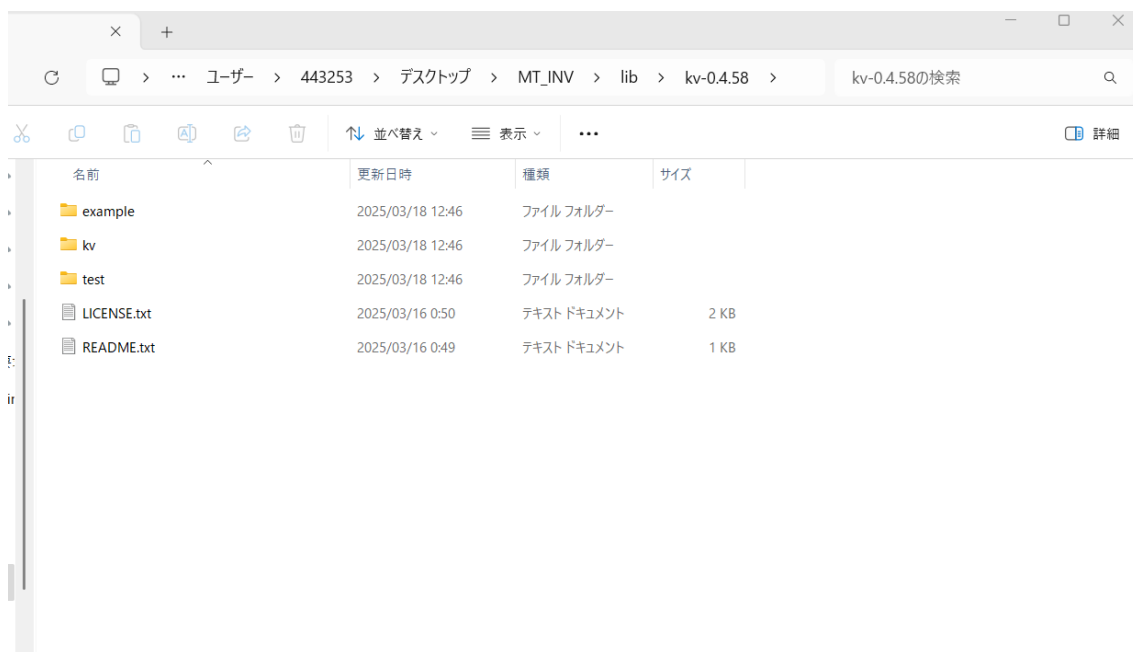
- ・ Boost のインストール。以下のサイトが参考になる。Visual Studio が必要かもしれない。

<https://www.boost.org/users/download/#live>

<https://www.kkaneko.jp/tools/win/boost.html>

- kv を以下からダウンロードし、適当なフォルダに置く。

<http://verifiedby.me/kv/>



2 コンパイル

- ・ 必要なら以下を参考に make をインストール

https://qiita.com/BARANCE_TW/items/c7ffdb311df84d47bddd

- ・ oneAPI をインストールしインテルコンパイラを入手

<https://www.intel.com/content/www/us/en/developer/tools/oneapi/base-toolkit-download.html>

コンパイラへパスを通す（デフォルトでは C:\Program Files

(x86)\Intel\oneAPI\compiler\2025.0\bin および C:\Program Files

(x86)\Intel\oneAPI\compiler\2025.0\lib)

- ・ ソースコードと一緒に入っている makefile のライブラリへのパスを各々設定する。

```
1 CC = icpx
2 FLAGS = -qopenmp -O3 -xhost
3 INCB = -I C:\boost\build\include\boost-1_88
4 INCE = -I C:\Users\xxxxx\Desktop\MT_INV\lib\eigen-3.4.0
5 INCO = -I C:\Users\xxxxx\Desktop\MT_INV\lib\optimlib\header_only_version
6 INCK = -I C:\Users\xxxxx\Desktop\MT_INV\lib\kv-0.4.58
7 OUT = MT_INV.exe
8 MT_INV: MT.o Analysis.o BiCGSafe.o Boundary.o DivergenceCorrection.o Element.o F
9 $(CC) -o $(OUT) MT.o Analysis.o BiCGSafe.o Boundary.o DivergenceCorrection.o
10
11 MT.o: MT.cpp
12 $(CC) -c MT.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
13 Analysis.o: Analysis.cpp
14 $(CC) -c Analysis.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
15 BiCGSafe.o: BiCGSafe.cpp
16 $(CC) -c BiCGSafe.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
17 Boundary.o: Boundary.cpp
18 $(CC) -c Boundary.cpp $(INCB) $(INCE) $(INCO) $(INCK) $(FLAGS)
19 DivergenceCorrection.o: DivergenceCorrection.cpp
```

- ・ コマンドプロンプトを立ち上げ make
- ・ MT_INV.exe が出来上がる。
- ・ コンパイルは Visual Studio を使ってもできる。経験的にはこちらのほうが高速。
- ・ icpx を使う場合、omp_set_nested が廃止された旨の WARNING が出る可能性があるが、その場合、MT.cpp を以下のように変える。

```

・ if (omp_get_max_threads() >= 2 * analysis->boundary->omega.size()) {
・     omp_set_nested(1);
・ }
・ else {
・     omp_set_nested(0);
・ }
・

```

↓↓↓

```

if (omp_get_max_threads() >= 2 * analysis->boundary->omega.size()) {

    omp_set_max_active_levels (2);

}

else {

    omp_set_max_active_levels (1);

}

```